

INSTITUCIÓN: UNIVERSIDAD BANCARIA DE MÉXICO

ASIGNATURA: DESARROLLO DE APLICACIONES PARA INTERNET AVANZADO II

DOCENTE: ROGELIO REYES REYES



PROYECTO ACADÉMICO

DESARROLLO DE UNA APLICACIÓN ESCOLAR INTELIGENTE

Gestión de inscripciones escolares

Metodología Scrum

EQUIPO DEL PROYECTO

Carlos Eduardo Martínez Morales

Daen Sánchez Marín

Fernando Pérez Acuautla

Fernando Aguilar Velázquez

Pedro Jair Suárez Flores

AGOSTO DE 2026

DESARROLLO DE UNA APLICACIÓN ESCOLAR INTELIGENTE PARA LA GESTIÓN DE INSCRIPCIONES

Índice general

Apartado	Contenido
Información general	Equipo, funciones, objetivo y problemática.
Etapa 1	Actores, UFP, cadena de valor y diagrama general.
Etapa 2	Arquitectura, tecnologías y decisiones de solución.
Etapa 3	Requisitos, modelo normalizado, migración y base de datos.
Etapa 4	Recursos, rutas, contratos, validaciones, permisos y Postman.
Etapa 5	Desarrollo realizado y estado técnico de la API.
Etapa 6	Validaciones ejecutadas y estrategia de pruebas.
Etapa 7	Front end demostrativo y experiencia por roles.
Etapa 8	Evidencias, instalación y control de entrega.
Conclusiones	Resultados, alcance real y continuidad del proyecto.
Anexos	Glosario, comandos y trazabilidad Scrum.

Integrantes y responsabilidades generales

Núm.	Integrante	Responsabilidad
1	Carlos Eduardo Martínez Morales	Scrum Master y desarrollador
2	Daen Sánchez Marín	Líder técnico y desarrollador
3	Fernando Pérez Acuautla	Desarrollador
4	Fernando Aguilar Velázquez	Desarrollador
5	Pedro Jair Suárez Flores	Responsable de pruebas

Objetivo

Desarrollar una aplicación web que permita administrar el proceso de inscripción de estudiantes a una institución educativa, aplicando metodologías de análisis, diseño y desarrollo de APIs. El equipo deberá seleccionar las tecnologías más adecuadas para resolver el problema y justificar cada decisión tomada. La solución deberá contemplar tanto aspectos funcionales como de seguridad, organización de datos y experiencia del usuario.

Problemática

Una institución educativa realiza actualmente las inscripciones de manera manual mediante hojas de cálculo y formularios impresos.

Esto provoca diversos problemas: inscripciones duplicadas, grupos saturados, captura incorrecta de información, dificultad para consultar el historial de alumnos, poca seguridad en el acceso al sistema y falta de reportes para la toma de decisiones.

La dirección escolar solicita el desarrollo de una aplicación inteligente que permita administrar este proceso.

El equipo deberá analizar el problema y proponer una solución.

1. Primera etapa. Análisis del problema

El equipo deberá comprender completamente el problema antes de comenzar a programar.

1.1 Identificación de actores

- Aspirante
- Coordinación de Admisiones
- Control Escolar
- Coordinador Académico
- Alumno
- Sistema

1.2 Matriz Actores–Responsabilidades

Actor	Responsabilidad
Aspirante	Captura sus datos, selecciona carrera y entrega su expediente digital.
Aspirante	Consulta el avance y sustituye solamente los documentos observados.
Coordinación de Admisiones	Revisa la información y los documentos del aspirante.
Coordinación de Admisiones	Emite dictamen de aceptación, rechazo u observación con motivo.
Coordinación de Admisiones	Solicita la notificación externa correspondiente al aspirante.
Control Escolar	Enrola al aspirante aceptado y genera su matrícula y acceso inicial.
Control Escolar	Administra estados, datos y antecedentes de los alumnos.
Control Escolar	Atiende los mensajes de carácter administrativo del alumno.
Coordinación Académica	Administra grupos dentro de su carrera y controla su capacidad.
Coordinación Académica	Asigna o cambia el grupo del alumno cuando exista disponibilidad.
Coordinación Académica	Atiende mensajes académicos dirigidos a un coordinador específico.
Alumno	Consulta su matrícula, estado, carrera, grupo, periodo y turno.
Alumno	Envía mensajes a Control Escolar o a un coordinador específico.
Sistema	Aplica integridad, historial, control de proceso activo y validación de cupo.
Sistema	Protege el acceso, registra notificaciones y conserva trazabilidad.

1.3 Unidades Funcionales de Proceso (Procesos Principales)

1. Pre-registro y Carga Documental de Aspirante

Elemento	Información
Nombre descriptivo	UFP-01: Pre-registro y Carga Documental de Aspirante
Qué hace	Proporciona un formulario público donde el aspirante captura sus datos personales, selecciona su carrera de interés y adjunta sus documentos. Al finalizar el envío, desencadena una notificación automática vía correo electrónico (EmailJS) confirmando la recepción del trámite.
Actores involucrados	Aspirante, Sistema (EmailJS).
Problemática que resuelve	Elimina las inscripciones manuales en papel/hojas de cálculo y evita la captura incorrecta de datos iniciales mediante validaciones en el formulario web.

2. Revisión, Validación y Dictamen de Admisiones

Elemento	Información
Nombre descriptivo	UFP-02: Revisión, Validación y Dictamen de Admisiones

Elemento	Información
Qué hace	Permite al equipo de admisiones cotejar los datos capturados contra los documentos adjuntos. Si detecta inconsistencias o faltantes, rechaza el trámite y notifica al aspirante por correo para que reenvíe sus datos; si todo es correcto, aprueba al aspirante y lo habilita para el enrolamiento oficial.
Actores involucrados	Coordinación de Admisiones, Aspirante, Sistema (EmailJS).
Problemática que resuelve	Previene el ingreso de información falsa o incompleta al sistema y automatiza el seguimiento de aspirantes sin depender de llamadas o trámites presenciales.

3. Enrolamiento Escolar y Generación de Credenciales Únicas

Elemento	Información
Nombre descriptivo	UFP-03: Enrolamiento Escolar y Generación de Credenciales Únicas
Qué hace	Registra formalmente al aspirante aceptado en la base de datos de la institución, generando automáticamente su matrícula única personalizada y una contraseña de acceso para que pueda iniciar sesión como alumno oficial.
Actores involucrados	Control Escolar, Sistema.
Problemática que resuelve	Errores en la asignación manual de matrículas y falta de credenciales individuales de acceso para los estudiantes.

4. Administración General de Alumnos y Coordinadores por Carrera

Elemento	Información
Nombre descriptivo	UFP-04: Administración General de Alumnos y Coordinadores por Carrera
Qué hace	Proporciona un módulo a Control Escolar con funciones CRUD para gestionar los expedientes de los alumnos y la asignación de coordinadores por carrera (ID_Coordinador, Nombre, ID_Carrera). Permite la búsqueda rápida de alumnos por nombre o matrícula única para corregir errores de captura.
Actores involucrados	Control Escolar, Sistema.
Problemática que resuelve	Dificultad para consultar, filtrar y actualizar el historial de alumnos, además de centralizar el control de las carreras y sus responsables.

5. Asignación Académica de Grupo y Control de Cupo por Coordinación

Elemento	Información
Nombre descriptivo	UFP-05: Asignación Académica de Grupo y Control de Cupo
Qué hace	Permite a Coordinación administrar grupos de su carrera, definir capacidad y asignar alumnos únicamente cuando exista disponibilidad. El catálogo de materias queda fuera del núcleo de esta versión.
Actores involucrados	Coordinación Académica, Sistema.
Problemática que resuelve	Desorganización en la distribución de alumnos y saturación de grupos.

6. Consulta de Información Personal, Estado e Inscripción

Elemento	Información
Nombre descriptivo	UFP-06: Consulta de Información Personal, Estado e Inscripción
Qué hace	Permite al alumno consultar su matrícula, estado, carrera, grupo, periodo y turno desde un portal privado.
Actores involucrados	Alumno, Sistema.
Problemática que resuelve	Reduce consultas presenciales y ofrece visibilidad inmediata de la situación escolar registrada.

7. Gestión de Solicitudes de Ayuda y Corrección de Datos

Elemento	Información
Nombre descriptivo	UFP-07: Gestión de Solicitudes de Ayuda y Corrección de Datos
Qué hace	Mantiene conversaciones internas. Los mensajes administrativos llegan a la bandeja compartida de Control Escolar y los académicos pueden dirigirse a un coordinador específico.
Actores involucrados	Alumno, Control Escolar, Coordinación Académica, Sistema.
Problemática que resuelve	Evita solicitudes informales y conserva un historial consultable de la atención.

8. Emisión de Reportes Estadísticos y Control de Seguridad

Elemento	Información
Nombre descriptivo	UFP-08: Historial, Indicadores y Control de Seguridad
Qué hace	Permite consultar procesos históricos por CURP y preparar indicadores de admisión e inscripción; la protección de acceso se define mediante autenticación y roles.
Actores involucrados	Control Escolar, Sistema.
Problemática que resuelve	Falta de trazabilidad y de información consolidada para el seguimiento administrativo.

1.4 Unidades Funcionales Sugeridas

1. Validación Automática de Disponibilidad y Cupo Máximo

Elemento	Información
Nombre descriptivo	UFP-SUG-01: Validación Automática de Disponibilidad y Cupo Máximo
Qué hace	Módulo que verifica en tiempo real la capacidad de un grupo antes de que el coordinador o Control Escolar confirme la asignación de un alumno, bloqueando el registro si el límite ha sido alcanzado.
Actores involucrados	Sistema, Coordinador Académico, Control Escolar.
Problemática que resuelve	Evita de forma directa la problemática de grupos saturados, impidiendo sobrecupos por descuido manual.

2. Autenticación, Control de Sesión y Roles de Usuario

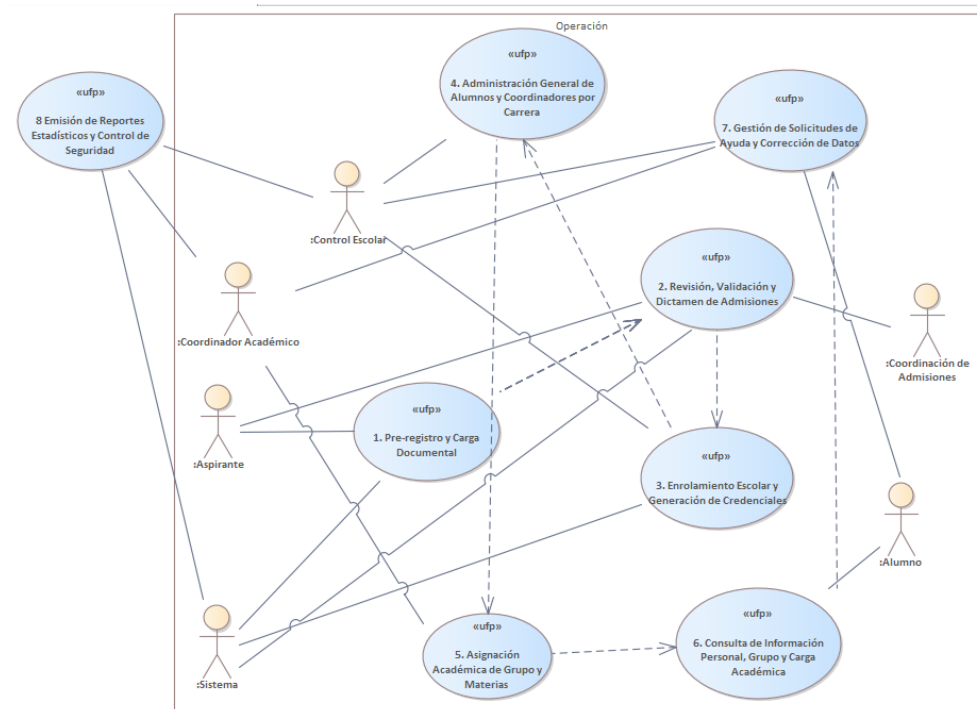
Elemento	Información
Nombre descriptivo	UFP-SUG-02: Autenticación, Control de Sesión y Roles de Usuario
Qué hace	Módulo técnico encargado de validar las credenciales (matrícula/contraseña para alumnos; credenciales de personal para administrativos), gestionar el cierre de sesión por inactividad y restringir vistas según el rol (Alumno, Control Escolar, Coordinador).
Actores involucrados	Alumno, Control Escolar, Coordinador Académico, Sistema.
Problemática que resuelve	Ataca directamente la poca seguridad en el acceso al sistema, impidiendo que un alumno acceda a módulos administrativos o a la información de otros estudiantes.

3. Reenvío de Información Reclamada por Admisiones

Elemento	Información
Nombre descriptivo	UFP-SUG-03: Subsanción y Re-evaluación Documental
Qué hace	Permite que un aspirante con expediente observado acceda mediante un token temporal y sustituya únicamente los documentos indicados, sin crear otro folio ni repetir el registro.
Actores involucrados	Aspirante, Coordinación de Admisiones, Sistema.
Problemática que resuelve	Evita folios duplicados y conserva la versión anterior como evidencia.

1.5 Cadena de valor y diagrama 0





2. Segunda etapa. Diseño de la solución

Se adoptó una arquitectura cliente-servidor con una API REST modular dentro de un monolito Node.js y Express. MySQL funciona como persistencia relacional mediante Prisma ORM. El mismo servidor publica actualmente una interfaz web estática de demostración; la integración de esa interfaz con los endpoints de negocio se realizará conforme avance la implementación.

2.1 Tecnologías registradas en el avance

Las tecnologías se registran de acuerdo con su uso real o previsto. La documentación distingue los componentes ya implementados de aquellos cuyo contrato fue aprobado pero todavía requiere desarrollo.

Tecnología	Uso dentro del proyecto	Justificación
Node.js 22 y Express 5	Servidor HTTP y API modular. Implementado.	Permiten ejecutar TypeScript compilado, publicar el front y separar rutas por módulo dentro de una base sencilla de mantener.
TypeScript 7	Lenguaje del backend. Implementado.	Aporta tipado estático, facilita refactorizaciones y detecta inconsistencias antes de desplegar.
MySQL 8.4	Persistencia relacional. Implementado localmente.	Sus relaciones, restricciones e índices son adecuados para identidad, expedientes, estados, inscripciones e historial.
Prisma ORM 7.9	Modelo, cliente, migración y datos semilla. Implementado.	Centraliza el esquema, genera acceso tipado y permite reproducir la estructura mediante migraciones.
Microsoft Azure	Alojamiento previsto de MySQL.	Centralizará la base sin alojar el repositorio ni sustituir el servicio de ejecución de la API.

Tecnología	Uso dentro del proyecto	Justificación
JWT	Autenticación y permisos. Diseñado; pendiente en backend.	El contrato permite sesiones sin estado en el servidor y separación de permisos por rol y carrera.
EmailJS	Notificaciones externas al aspirante. Diseñado; pendiente de integración.	Se limita a confirmaciones, observaciones y dictámenes del proceso de admisión.
CORS y dotenv	Orígenes y variables de entorno. Implementados.	Evitan valores sensibles en el código y permiten adaptar el servidor a distintos entornos.
HTML5, CSS3 y Bootstrap 5.3	Interfaz responsive. Implementada como demostración.	Ofrecen formularios y paneles claros sin introducir un framework de front adicional.
jsPDF 4.2	PDF de acceso inicial. Implementado en la demostración.	Genera localmente el comprobante con matrícula y contraseña temporal al concluir el enrolamiento demostrativo.
Git y GitHub	Control de versiones y repositorio privado. Implementados.	Mantienen historial, ramas y revisión del código sin publicar credenciales ni datos reales.
Postman	Contrato y pruebas de API. Colección implementada.	Agrupar 35 solicitudes y prepara la ejecución reproducible cuando los endpoints de negocio estén disponibles.

2.2 Relación entre las tecnologías

Microsoft Azure se reserva exclusivamente para alojar MySQL. El código se conserva en el repositorio privado de GitHub; el servicio de ejecución de la API podrá ser Render u otra plataforma compatible con Node.js, siempre que utilice variables de entorno y una conexión segura a la base alojada.

3. Tercera etapa. Diseño e implementación de la base de datos

Estado de la etapa. SCRUM-13 a SCRUM-17 se consideran concluidos para la línea base actual. El modelo fue normalizado, convertido en un esquema Prisma, materializado en una migración MySQL y acompañado por catálogos iniciales.

El alcance prioritario permanece en admisión e inscripción. No se incorporó una entidad de materias porque ese catálogo es complementario y no condiciona el flujo principal aprobado.

3.1 Propósito del diseño de la base de datos

Diseñar una persistencia relacional que registre cada intento de admisión, mantenga la identidad única por CURP, conserve expedientes y documentos históricos, permita el enrolamiento y la inscripción, controle cupos y soporte la comunicación institucional sin borrar información relevante.

3.2 Objetivos de la base de datos

Tabla 3.1. Objetivos del modelo de datos.

Objetivo	Descripción
Centralización	Concentrar identidad, admisión, enrolamiento, inscripción e historial en una sola estructura relacional.
Integridad	Aplicar claves, restricciones e índices que impidan referencias inválidas y duplicidades funcionales.
Historial por CURP	Conservar varios procesos de una persona y permitir solamente uno activo a la vez.
Documentos	Mantener versiones documentales y señalar una única versión vigente por tipo y expediente.
Seguridad	Separar cuentas, roles y credenciales; almacenar únicamente hashes de contraseña.
Cupo	Relacionar grupo, periodo e inscripción y calcular disponibilidad mediante estados que ocupan lugar.
Mensajería	Conservar conversaciones, remitente real y lecturas por usuario.
Reproducibilidad	Poder reconstruir la base mediante migraciones y datos semilla.

3.3 Análisis de requisitos y entidades (SCRUM-13)

3.3.1 Requisitos funcionales

Tabla 3.2. Requisitos funcionales consolidados.

ID	Requisito funcional	Actor
RF-01	Registrar la identidad de la persona y crear un intento de admisión con carrera y datos de contacto.	Aspirante / Sistema
RF-02	Generar folio y expediente; recibir documentos por tipo y versión.	Sistema / Aspirante
RF-03	Revisar documentos, registrar observaciones y emitir dictamen.	Admisiones
RF-04	Permitir la subsanación sin duplicar folio y conservar la versión sustituida.	Aspirante / Admisiones

ID	Requisito funcional	Actor
RF-05	Impedir dos procesos activos para una misma CURP y conservar procesos cerrados.	Sistema
RF-06	Enrolar un expediente aceptado y generar una matrícula única.	Control Escolar / Sistema
RF-07	Administrar estados de alumno y registrar su historial.	Control Escolar
RF-08	Administrar carreras, periodos y grupos con capacidad máxima.	Control Escolar / Coordinación
RF-09	Inscribir un alumno en un grupo compatible sin superar su cupo.	Control Escolar / Coordinación
RF-10	Consultar perfil e inscripción propios desde el portal del alumno.	Alumno
RF-11	Crear conversaciones y mensajes dirigidos a Control Escolar o a Coordinación.	Alumno / Personal
RF-12	Registrar las notificaciones externas enviadas al aspirante.	Sistema
RF-13	Consultar el historial relacionado con una CURP sin eliminar procesos anteriores.	Control Escolar

3.3.2 Requisitos no funcionales

Tabla 3.3. Requisitos no funcionales.

ID	Requisito no funcional
RNF-01	MySQL debe mantener integridad referencial y restringir el borrado de información histórica.
RNF-02	Las contraseñas se conservarán como hashes; nunca se devolverán ni almacenarán en texto plano.
RNF-03	El acceso definitivo se protegerá por JWT y permisos de rol.
RNF-04	Las operaciones críticas se ejecutarán en transacciones para evitar estados parciales.
RNF-05	El esquema, la migración y los catálogos deberán ser reproducibles desde el repositorio.
RNF-06	La configuración y las credenciales se suministrarán mediante variables de entorno.
RNF-07	La API utilizará respuestas JSON uniformes y no expondrá detalles internos de Prisma o MySQL.
RNF-08	Los datos y archivos de demostración serán ficticios; no se publicarán secretos ni documentos reales.

3.4 Reglas de datos e integridad

Tabla 3.4. Reglas consolidadas de integridad.

ID	Regla de datos
RN-01	La CURP es única en tbl_persona; los intentos históricos se registran en tbl_aspirante.
RN-02	tbl_proceso_activo permite un solo proceso vigente por persona.
RN-03	Cada aspirante tiene como máximo un expediente y cada expediente puede originar como máximo un alumno.
RN-04	Solo un expediente aceptado puede continuar al enrolamiento.

ID	Regla de datos
RN-05	La matrícula es única de forma global en tbl_usuario.
RN-06	Cada versión documental es única por expediente, tipo y número de versión.
RN-07	tbl_documento_vigente identifica una sola versión actual por expediente y tipo documental.
RN-08	Cada grupo pertenece a una carrera y a un periodo.
RN-09	Cada alumno tiene como máximo una inscripción por periodo.
RN-10	El grupo y la inscripción deben pertenecer al mismo periodo mediante una relación compuesta.
RN-11	El cupo se calcula con inscripciones cuyo estado tenga ocupa_cupo = true.
RN-12	Los cambios de expediente, alumno e inscripción se registran en tablas históricas.
RN-13	Los registros históricos utilizan ON DELETE RESTRICT y se cierran mediante estados.
RN-14	Una conversación se dirige al departamento de Control Escolar o a un coordinador autorizado.
RN-15	Cada lectura de mensaje es única por mensaje y usuario.

3.5 Modelo lógico definitivo

Tabla 3.5. Catálogo de las 25 tablas implementadas.

Dominio	Tabla física	Responsabilidad
Identidad	tbl_persona	Identidad única, CURP, nombre y contacto principal.
Identidad	tbl_aspirante	Cada intento de ingreso asociado con persona y carrera.
Identidad	tbl_proceso_activo	Apuntador único al intento vigente de una persona.
Admisiones	tbl_expediente	Folio, estado, observaciones y dictamen.
Admisiones	tbl_cat_estado_expediente	Estados, orden, bloqueo de CURP y condición terminal.
Documentos	tbl_cat_tipo_documento	Tipos institucionales, obligatoriedad y vigencia.
Documentos	tbl_documento	Metadatos, revisión y versiones del archivo.
Documentos	tbl_documento_vigente	Versión actual por expediente y tipo.
Seguridad	tbl_usuario	Matrícula, hash, tipo de usuario y estado de acceso.
Alumnos	tbl_alumno	Alumno originado por un expediente aceptado.
Alumnos	tbl_cat_estado_alumno	Activo, bajas y egreso con reglas de vigencia.
Personal	tbl_empleado	Cuenta del personal, rol y carrera opcional.
Personal	tbl_cat_rol_empleado	Admisiones, Control Escolar y Coordinación.
Oferta	tbl_cat_carrera	Carreras activas y sus relaciones.
Oferta	tbl_cat_periodo	Ciclo escolar, fechas y estado.
Oferta	tbl_grupo	Clave, turno, grado, capacidad, carrera y periodo.
Inscripción	tbl_inscripcion	Relación alumno-grupo-periodo y responsable.

Dominio	Tabla física	Responsabilidad
Inscripción	tbl_cat_estado_inscripcion	Estados que indican si una inscripción ocupa cupo.
Historial	tbl_historial_inscripcion	Cambios de grupo y estado de inscripción.
Historial	tbl_historial_estado_expediente	Transiciones del expediente.
Historial	tbl_historial_estado_alumno	Transiciones académicas del alumno.
Mensajería	tbl_conversacion	Hilo, área destino, responsable y estado.
Mensajería	tbl_mensaje	Contenido, remitente real y fecha de envío.
Mensajería	tbl_mensaje_lectura	Lectura por mensaje y usuario.
Notificaciones	tbl_notificacion_aspirante	Evidencia del envío externo y su resultado.

3.5.1 Identidad e historial por CURP

La identidad se concentra en `tbl_persona`, donde la CURP es única. Cada nuevo intento se registra en `tbl_aspirante`, por lo que una persona puede conservar varios procesos históricos sin duplicar sus datos principales. `tbl_proceso_activo` funciona como autoridad para impedir dos procesos simultáneos.

3.5.2 Expediente y control documental

El expediente conserva el folio y el dictamen. Los archivos se modelan como registros independientes; cada sustitución crea una versión y `tbl_documento_vigente` señala la versión que debe mostrar la aplicación. La versión anterior permanece disponible como evidencia.

3.5.3 Cuenta de acceso y roles

`tbl_usuario` centraliza la matrícula y el hash de contraseña para alumnos y empleados. La especialización uno a uno evita matrículas duplicadas y permite que el rol administrativo y la carrera de Coordinación residan en entidades controladas.

3.5.4 Periodos, grupos e inscripciones

La inscripción referencia al alumno, el periodo, el grupo, el estado y el empleado responsable. Una clave foránea compuesta obliga a que grupo e inscripción compartan periodo. El límite de cupo se valida de forma transaccional al contar únicamente estados que ocupan lugar.

3.5.5 Mensajería y notificaciones

La mensajería interna separa conversaciones, mensajes y lecturas. EmailJS no reemplaza este módulo: las notificaciones externas al aspirante se registran aparte para conservar tipo, destinatario, estado e identificador externo.

3.6 Relaciones y cardinalidades

Tabla 3.6. Relaciones principales del modelo.

Origen	Cardinalidad	Destino	Interpretación
tbl_persona	1:N	tbl_aspirante	Una identidad puede realizar varios intentos históricos.

Origen	Cardinalidad	Destino	Interpretación
tbl_persona	1:0..1	tbl_proceso_activo	Solo puede existir un intento vigente por persona.
tbl_aspirante	1:0..1	tbl_expediente	Un intento origina como máximo un expediente.
tbl_expediente	1:N	tbl_documento	El expediente contiene versiones documentales.
tbl_expediente	1:0..1	tbl_alumno	Solo el proceso aceptado puede originar un alumno.
tbl_usuario	1:0..1	tbl_alumno / tbl_empleado	Una cuenta se especializa en un solo tipo.
tbl_cat_carrera	1:N	aspirantes, alumnos, empleados y grupos	La carrera se reutiliza sin duplicar texto.
tbl_cat_periodo	1:N	tbl_grupo	Un periodo ofrece varios grupos.
tbl_alumno	1:N	tbl_inscripcion	El alumno conserva su trayectoria por periodo.
tbl_grupo	1:N	tbl_inscripcion	El grupo recibe alumnos hasta su capacidad.
tbl_inscripcion	1:N	tbl_historial_inscripcion	Cada movimiento queda registrado.
tbl_alumno	1:N	tbl_conversacion	El alumno puede iniciar varios hilos.
tbl_conversacion	1:N	tbl_mensaje	Cada hilo contiene múltiples intervenciones.
tbl_mensaje	N:M	tbl_usuario	tbl_mensaje_lectura resuelve la lectura por usuario.
tbl_aspirante	1:N	tbl_notificacion_aspirante	Se conserva el historial de envíos externos.

3.7 Criterios de integridad y vigencia

- La baja temporal mantiene el proceso activo y conserva bloqueada la CURP.
- El rechazo, la cancelación, la baja definitiva y el egreso permiten liberar tbl_proceso_activo sin eliminar antecedentes.
- Los catálogos incluyen banderas como bloquea_curp, es_terminal u ocupa_cupo para evitar condiciones codificadas como texto libre.
- Las operaciones de historial utilizan llaves foráneas y fechas, no sobrescrituras destructivas.
- Las relaciones históricas se protegen principalmente con ON DELETE RESTRICT.

3.8 Casos de validación del modelo

Tabla 3.7. Casos de validación del modelo.

Caso	Resultado	Fundamento
CURP con proceso activo	Bloqueado	tbl_proceso_activo ya contiene a la persona.

Caso	Resultado	Fundamento
Solicitud rechazada y nuevo intento	Permitido	Se elimina solo el apuntador activo; el expediente anterior permanece.
Baja temporal y nuevo intento	Bloqueado	La relación académica sigue vigente.
Baja definitiva y nueva carrera	Permitido	Se crea otro aspirante para la misma persona y se conserva todo el historial.
Grupo al límite	Bloqueado	La transacción cuenta inscripciones que ocupan cupo antes de insertar.
Documento observado	Sustitución controlada	Se inserta una nueva versión y se actualiza tbl_documento_vigente.

3.9 Trazabilidad entre requisitos y entidades

Tabla 3.8. Trazabilidad funcional del modelo.

Proceso	Entidades	Cobertura
Admisión	persona, aspirante, proceso_activo, expediente	Identidad, folio, vigencia y dictamen.
Documentación	tipo_documento, documento, documento_vigente	Tipos, versiones y archivo actual.
Enrolamiento	usuario, alumno, estado_alumno	Matrícula y condición académica.
Control de acceso	usuario, empleado, rol_empleado	Credenciales y permisos.
Inscripción	periodo, grupo, inscripción, estado_inscripción	Asignación y cupo.
Historial	tres tablas de historial	Transiciones auditables.
Mensajería	conversación, mensaje, mensaje_lectura	Atención interna y lecturas.
Correo a aspirante	notificación_aspirante	Evidencia de EmailJS.

3.10 Criterios del diseño físico

- Nombres físicos en snake_case con prefijo tbl_ y catálogos identificados como tbl_cat_*.
- Identificadores enteros sin signo y fechas con precisión de milisegundos cuando se requiere trazabilidad.
- Restricciones UNIQUE para CURP, folio, matrícula, claves de catálogo y combinaciones funcionales.
- Índices para nombres, estados, fechas, carrera, periodo, cupo, bandejas de mensajes e historial.
- El borrado físico se evita en registros con valor histórico; los estados expresan la vigencia.

3.10.1 Diagrama entidad–relación de la base de datos

La figura representa las 25 tablas del modelo físico definido en Prisma. Para conservar su legibilidad, muestra las claves primarias, foráneas y únicas más relevantes, junto con las cardinalidades esenciales; los atributos completos, restricciones e índices permanecen documentados en el esquema y la migración SQL.



Figura 3.1. Diagrama entidad–relación del modelo físico AUT-INS.

3.11 Primera Forma Normal (SCRUM-14)

La Primera Forma Normal exige que cada atributo contenga un valor atómico y que no existan grupos repetidos dentro de una fila. El modelo inicial almacenaba documentos como columnas del expediente y combinaba varios datos de identidad o lectura en una sola entidad; esas estructuras fueron separadas.

Tabla 3.9. Aplicación de la Primera Forma Normal.

Situación analizada	Corrección aplicada
Documentos como columnas del expediente	tbl_documento registra una fila por archivo, tipo y versión.
Varios procesos dentro del mismo registro personal	tbl_persona se separa de tbl_aspirante y cada intento ocupa una fila.
Estado o rol capturado como texto libre	Se usan catálogos o enumeraciones con un solo valor por atributo.
Lista de mensajes dentro de una conversación	Cada mensaje es una fila de tbl_mensaje.
Lectura global incrustada en mensaje	tbl_mensaje_lectura registra una fila por mensaje y usuario.
Nombre completo sin estructura	Nombre, apellido paterno y apellido materno se almacenan por separado.

Resultado de 1FN. Todas las tablas poseen filas identificables, valores atómicos y ausencia de grupos repetidos. Los archivos reales permanecen fuera de las columnas relacionales; la base conserva únicamente sus metadatos y ubicación.

3.12 Segunda Forma Normal (SCRUM-15)

La Segunda Forma Normal requiere cumplir 1FN y que cada atributo no clave dependa de la clave completa. La mayoría de las tablas utiliza una clave primaria simple, por lo que no puede presentar dependencias parciales. La revisión se concentró en las claves compuestas y combinaciones únicas.

Tabla 3.10. Revisión de dependencias completas en 2FN.

Tabla	Clave revisada	Comprobación
tbl_documento_vigente	(id_expediente, id_tipo_documento)	id_documento depende de ambos valores: identifica la versión vigente de ese tipo dentro de ese expediente.
tbl_mensaje_lectura	(id_mensaje, id_usuario)	leido_en corresponde a la lectura de un mensaje por un usuario específico.
tbl_documento	UNIQUE expediente + tipo + versión	Los atributos del archivo dependen de la fila documental, no solo del expediente o del tipo.
tbl_grupo	UNIQUE periodo + carrera + clave	Los datos del grupo se conservan en tbl_grupo; carrera y periodo se resuelven mediante sus catálogos.
tbl_inscripcion	UNIQUE alumno + periodo	La fila representa la inscripción de ese alumno durante un periodo; el grupo compatible se valida por relación compuesta.

Resultado de 2FN. No existen atributos que dependan únicamente de una parte de una clave compuesta. Los datos descriptivos de tipos, estados, carreras y periodos residen en sus propias tablas.

3.13 Tercera Forma Normal (SCRUM-16)

La Tercera Forma Normal exige cumplir 2FN y eliminar dependencias transitivas entre atributos no clave. Se separaron las descripciones reutilizables y las responsabilidades que podrían depender indirectamente de otra entidad.

Tabla 3.11. Eliminación de dependencias transitivas en 3FN.

Dependencia detectada	Entidad separada	Resultado
CURP y datos personales repetidos en cada trámite	tbl_persona	El aspirante referencia la identidad; no duplica sus atributos.
Carrera escrita en aspirante, alumno, empleado y grupo	tbl_cat_carrera	Cada entidad conserva únicamente id_carrera.
Contraseña en alumno y empleado	tbl_usuario	La cuenta concentra matrícula, hash, tipo y vigencia.
Descripción y reglas del estado en cada expediente	tbl_cat_estado_expediente	El expediente guarda id_estado_expediente.
Descripción del estado académico	tbl_cat_estado_alumno	El alumno guarda id_estado_alumno.
Regla de ocupación de cupo repetida	tbl_cat_estado_inscripcion	La bandera ocupa_cupo pertenece al estado, no a cada inscripción.
Datos del archivo actual repetidos	tbl_documento_vigente	El apuntador vigente evita duplicar metadatos.
Nombre del remitente dentro del mensaje	tbl_usuario	tbl_mensaje conserva id_usuario_remitente.

Resultado de 3FN. Las descripciones y reglas compartidas dependen de sus propias claves. El esquema evita que un cambio de carrera, rol o estado obligue a actualizar textos repetidos en múltiples tablas.

3.14 Diseño e implementación de la base de datos (SCRUM-17)

3.14.1 Archivos implementados

Tabla 3.12. Evidencias de implementación de la base.

Archivo	Responsabilidad
prisma/schema.prisma	25 modelos, 8 enumeraciones, relaciones, restricciones e índices.
prisma/migrations/20260809000000_init/migration.sql	Migración inicial que crea las tablas y claves foráneas de MySQL.
prisma/seed.ts	Carga idempotente de estados, roles y tipos documentales.
prisma.config.ts	Configuración de Prisma 7 y lectura segura de DATABASE_URL.
src/config/database.ts	Cliente Prisma compartido mediante el adaptador MariaDB/MySQL.
scripts/local-mysql.ps1	Preparación de una instancia local aislada para desarrollo.

3.14.2 Catálogos iniciales

Tabla 3.13. Catálogos cargados de forma idempotente.

Catálogo	Valores semilla
Estado de expediente	PENDIENTE_DOCUMENTOS, PENDIENTE_REVISION, OBSERVADO, ACEPTADO, RECHAZADO, CANCELADO
Estado de alumno	ACTIVO, BAJA_TEMPORAL, BAJA_DEFINITIVA, EGRESADO
Estado de inscripción	ACTIVA, CANCELADA, FINALIZADA
Roles	ADMISIONES, CONTROL_ESCOLAR, COORDINACION
Tipos documentales	Acta, certificado, identificación, comprobante, CURP y fotografía

3.14.3 Operaciones transaccionales

- Pre-registro: persona, aspirante, expediente y proceso activo.
- Dictamen: estado, historial, notificación y liberación del proceso cuando sea terminal.
- Enrolamiento: usuario, alumno e historial inicial.
- Inscripción y cambio de grupo: bloqueo, validación de carrera y periodo, conteo de cupo y movimiento histórico.
- Sustitución documental: nueva versión, actualización del apuntador vigente y conservación de la anterior.
- Baja definitiva o egreso: estado, historial y liberación controlada del proceso activo.

3.14.4 Consultas de referencia

Consultar el proceso vigente de una CURP:

```
SELECT p.curp, e.folio, ce.codigo AS estado
FROM tbl_persona p
JOIN tbl_proceso_activo pa ON pa.id_persona = p.id_persona
JOIN tbl_aspirante a ON a.id_aspirante = pa.id_aspirante
JOIN tbl_expediente e ON e.id_aspirante = a.id_aspirante
JOIN tbl_cat_estado_expediente ce ON ce.id_estado_expediente = e.id_estado_expediente
WHERE p.curp = ?;
```

Consultar el historial completo de una CURP:

```
SELECT a.fecha_registro, e.folio, c.nombre AS carrera, ce.codigo AS estado
FROM tbl_persona p
JOIN tbl_aspirante a ON a.id_persona = p.id_persona
JOIN tbl_expediente e ON e.id_aspirante = a.id_aspirante
JOIN tbl_cat_carrera c ON c.id_carrera = a.id_carrera
JOIN tbl_cat_estado_expediente ce ON ce.id_estado_expediente = e.id_estado_expediente
WHERE p.curp = ? ORDER BY a.fecha_registro;
```

Calcular cupo ocupado:

```
SELECT g.capacidad_maxima, COUNT(i.id_inscripcion) AS ocupados
FROM tbl_grupo g
LEFT JOIN tbl_inscripcion i ON i.id_grupo = g.id_grupo
LEFT JOIN tbl_cat_estado_inscripcion ei
ON ei.id_estado_inscripcion = i.id_estado_inscripcion AND ei.occupa_cupo = TRUE
WHERE g.id_grupo = ? GROUP BY g.id_grupo, g.capacidad_maxima;
```

3.14.5 Validación y ejecución

```
npm run mysql:setup
npm run db:validate
npm run db:deploy
npm run db:seed
npm run db:studio
```

Resultado de SCRUM-17. El esquema fue validado con Prisma, la migración forma parte del repositorio y los catálogos pueden cargarse nuevamente sin crear duplicados. La conexión productiva a Azure deberá suministrarse mediante DATABASE_URL; no se incluyen credenciales en el documento ni en Git.

4. Cuarta etapa. Diseño de la API

Estado del apartado 4. Los resultados de SCRUM-19 a SCRUM-24 se encuentran integrados y revisados. La etapa queda concluida como línea base para iniciar el desarrollo; cualquier modificación posterior deberá registrarse como cambio de contrato antes de aplicarse al código.

Tabla 4.1. Control de entregables de la etapa.

Actividad	Entregable integrado	Estado
SCRUM-19	Identificación y delimitación de recursos	Concluido
SCRUM-20	Rutas y métodos HTTP	Concluido
SCRUM-21	Entradas, salidas y respuestas	Concluido
SCRUM-22	Validaciones, excepciones y códigos HTTP	Concluido
SCRUM-23	Autenticación JWT y permisos por rol	Concluido
SCRUM-24	Colección inicial de Postman y revisión integral	Concluido

4.1 Identificación de recursos de la API (SCRUM-19)

Los recursos se derivan de las UFP y se agrupan por capacidad funcional. La estructura no replica las tablas de MySQL: la API expone contratos orientados al proceso y mantiene separadas las responsabilidades de Admisiones, Control Escolar, Coordinación y Alumno.

Tabla 4.2. Recursos funcionales de la API REST.

Recurso	Responsabilidad	UFP	Alcance
/auth	Inicio de sesión, emisión y validación de JWT.	UFP-SUG-02	Núcleo
/aspirantes	Pre-registro, consulta del trámite y datos de contacto.	UFP-01	Núcleo
/expedientes	Revisión, observaciones, dictamen y subsanación.	UFP-02 y UFP-SUG-03	Núcleo
/documentos	Carga y reemplazo controlado del expediente digital.	UFP-01 y UFP-02	Núcleo
/alumnos	Enrolamiento, matrícula, estado y consulta personal.	UFP-03, UFP-04 y UFP-06	Núcleo
/empleados	Cuentas del personal y relación con rol y carrera.	UFP-04	Apoyo
/carreras	Oferta educativa utilizada durante admisión e inscripción.	UFP-04 y UFP-05	Apoyo
/periodos	Periodos habilitados para grupos e inscripciones.	UFP-05	Apoyo
/grupos	Grupos, turno, carrera, periodo y capacidad máxima.	UFP-05 y UFP-SUG-01	Núcleo
/inscripciones	Relación entre alumno y grupo con validación de cupo.	UFP-03, UFP-05 y UFP-SUG-01	Núcleo
/conversaciones	Hilos de atención interna del alumno.	UFP-07	Complementario

Recurso	Responsabilidad	UFP	Alcance
/mensajes	Intervenciones y seguimiento de lectura de cada hilo.	UFP-07	Complementario
/historial	Consulta de procesos anteriores asociados con una CURP.	UFP-04 y UFP-08	Apoyo
/reportes	Indicadores de admisión, inscripción y ocupación.	UFP-08	Complementario
/materias	Consulta académica ampliada, sin bloquear la primera entrega.	UFP-05 y UFP-06	Complementario

4.2 Diseño de rutas (SCRUM-20)

La ruta base será /api. Los nombres se expresan en plural, los identificadores variables se representan con :id y las acciones que no corresponden a un CRUD directo se modelan mediante subrecursos, por ejemplo dictamen, estado y lectura.

Tabla 4.3. Matriz consolidada de endpoints.

Módulo	Método	Ruta	Propósito	Acceso
Auth	POST	/api/auth/login	Validar credenciales y emitir JWT.	Público
Carreras	GET	/api/carreras	Consultar oferta activa para el registro.	Público
Aspirantes	POST	/api/aspirantes	Crear el pre-registro y folio.	Público
Aspirantes	GET	/api/aspirantes/:id/estado	Consultar el estado del trámite.	Token temporal
Documentos	POST	/api/aspirantes/:id/documentos	Adjuntar documentos iniciales.	Token temporal
Documentos	PATCH	/api/documentos/:id	Reemplazar un documento observado.	Token temporal
Expedientes	GET	/api/expedientes	Listar expedientes con filtros y paginación.	Admisiones / CE
Expedientes	GET	/api/expedientes/:id	Consultar datos y documentos de un expediente.	Admisiones / CE
Expedientes	PATCH	/api/expedientes/:id/dictamen	Aceptar, rechazar u observar la solicitud.	Admisiones
Alumnos	POST	/api/alumnos	Enrolar desde un expediente aceptado.	Control Escolar
Alumnos	GET	/api/alumnos	Listar alumnos según ámbito autorizado.	CE / Coordinación
Alumnos	GET	/api/alumnos/:id	Consultar expediente esencial del alumno.	CE / Coordinación
Alumnos	PATCH	/api/alumnos/:id/estado	Registrar baja, reincorporación o egreso.	Control Escolar
Portal	GET	/api/alumnos/me	Consultar perfil, matrícula y estado propios.	Alumno
Portal	GET	/api/alumnos/me/inscripcion	Consultar carrera, grupo, turno y periodo propios.	Alumno
Empleados	GET	/api/empleados	Consultar personal y roles.	Control Escolar
Empleados	POST	/api/empleados	Registrar personal autorizado.	Control Escolar
Empleados	PUT	/api/empleados/:id	Actualizar cuenta, rol o carrera asignada.	Control Escolar
Periodos	GET	/api/periodos	Consultar periodos disponibles.	CE / Coordinación
Periodos	POST	/api/periodos	Registrar un periodo institucional.	Control Escolar
Grupos	GET	/api/grupos	Consultar grupos y disponibilidad.	CE / Coordinación

Módulo	Método	Ruta	Propósito	Acceso
Grupos	POST	/api/grupos	Crear un grupo y establecer capacidad.	CE / Coordinación
Grupos	PUT	/api/grupos/:id	Actualizar datos autorizados del grupo.	CE / Coordinación
Inscripciones	POST	/api/inscripciones	Inscribir al alumno en un grupo con cupo.	CE / Coordinación
Inscripciones	GET	/api/inscripciones	Consultar inscripciones mediante filtros.	CE / Coordinación
Conversaciones	GET	/api/conversaciones	Listar hilos visibles para el usuario.	Usuario autenticado
Conversaciones	POST	/api/conversaciones	Crear una solicitud de ayuda.	Alumno
Conversaciones	GET	/api/conversaciones/:id	Consultar un hilo autorizado.	Participantes
Mensajes	GET	/api/conversaciones/:id/mensajes	Consultar mensajes sin modificar su estado.	Participantes
Mensajes	POST	/api/conversaciones/:id/mensajes	Enviar un mensaje en un hilo abierto.	Participantes
Mensajes	PATCH	/api/conversaciones/:id/lectura	Marcar como leídos los mensajes recibidos.	Participantes
Conversaciones	PATCH	/api/conversaciones/:id/estado	Cerrar o cambiar el estado del hilo.	Área responsable
Historial	GET	/api/historial/curp/:curp	Consultar procesos históricos de una persona.	Control Escolar
Reportes	GET	/api/reportes/inscripciones	Obtener indicadores del proceso.	Control Escolar
Materias	GET	/api/materias	Consultar catálogo académico si se habilita.	Autenticado

Nota de alcance. Materias y reportes se conservan como recursos complementarios; su desarrollo no condiciona la entrega funcional del proceso de admisión e inscripción.

4.3 Definición de entradas y respuestas (SCRUM-21)

Los cuerpos JSON utilizarán nombres en camelCase y no expondrán nombres de tablas, columnas internas, contraseñas, hashes ni detalles de Prisma. Las fechas se devolverán en formato ISO 8601 y los archivos se enviarán mediante multipart/form-data.

4.3.1 Formato uniforme de respuesta

Respuesta exitosa:

```
{
  "success": true,
  "message": "Operación realizada correctamente.",
  "data": {},
  "meta": { "timestamp": "2026-08-08T15:25:00Z" }
}
```

Respuesta de error:

```
{
  "success": false,
  "error": { "code": "BUSINESS_CONFLICT", "message": "Descripción del error.", "details": [] },
  "meta": { "timestamp": "2026-08-08T15:25:00Z" }
}
```

Las respuestas paginadas agregarán data.items y meta.pagination con page, limit, total y totalPages. Los valores predeterminados serán page=1 y limit=10; limit no podrá exceder 100.

4.3.2 Contratos principales

Tabla 4.4. Entradas y salidas de los contratos principales.

Ruta	Entrada	Salida esperada
POST /auth/login	matricula, password	token, tipoUsuario, rol y datos mínimos de sesión
POST /aspirantes	nombre, apellidos, curp, correo, telefono, carrerald	aspiranteld, expedienteld, folio, estado y token temporal
GET /aspirantes/:id/estado	Path :id y token temporal	folio, estado, observaciones permitidas y fechaActualizacion
POST /aspirantes/:id/documentos	archivo, tipoDocumento; PDF/JPG/PNG	documentold, nombre, tipo, estado y fechaCarga
PATCH /documentos/:id	archivo sustituto y token temporal	documentold, estado actualizado y fechaCarga
GET /expedientes	estado, carrerald, page, limit	lista paginada de expedientes autorizados
PATCH /expedientes/:id/dictamen	estado y observaciones cuando correspondan	expedienteld, estado, observaciones y fechaDictamen
POST /alumnos	expedienteld	alumnold, matricula, estado y carrera heredada
PATCH /alumnos/:id/estado	estado y motivo	alumnold, estadoAnterior, estadoActual y fechaActualizacion
GET /alumnos/me	JWT	matricula, nombre, carrera y estados permitidos
GET /alumnos/me/inscripcion	JWT	periodo, grupo, turno y situación de inscripción
POST /empleados	nombre, apellidos, matricula, rolld y carrerald condicional	empleadold, matricula, rol y carrera asignada
POST /periodos	nombre, fechaInicio, fechaFin y estado	periodold y datos registrados
POST /grupos	turno, grado, capacidadMaxima, carrerald y periodold	grupold, capacidad y cupoDisponible
POST /inscripciones	alumnold y grupold	inscripcionld, alumno, grupo, periodo y fechaInscripcion
POST /conversaciones	asunto y areaDestino	conversacionld, estado y fechaCreacion
POST /conversaciones/:id/mensajes	contenido; remitente derivado del JWT	mensajeld, conversacionld y fechaEnvio
PATCH /conversaciones/:id/lectura	Sin cuerpo obligatorio	cantidad de mensajes marcados y fechaLectura
GET /historial/curp/:curp	CURP en path	procesos históricos, estados y referencias relacionadas
GET /reportes/inscripciones	periodold y carrerald opcionales	totales y distribución del proceso

Los identificadores de alumno, empleado, conversación y remitente no se aceptarán desde el cliente cuando puedan obtenerse de la sesión. Esta medida evita que un usuario intente operar en nombre de otra persona.

4.4 Diseño de validaciones y errores (SCRUM-22)

Las validaciones se aplicarán antes de ejecutar cambios persistentes. Los conflictos funcionales devolverán mensajes comprensibles y códigos estables; los detalles internos de base de datos se registrarán solamente en los logs del servidor.

Tabla 4.5. Matriz de reglas, excepciones y códigos HTTP.

ID	Condición	Respuesta de la API	HTTP
VAL-01	Campos obligatorios ausentes o formato inválido.	VALIDATION_ERROR - Revise los datos enviados.	400
VAL-02	CURP distinta de 18 caracteres o con formato inválido.	INVALID_CURP - La CURP proporcionada no es válida.	400
VAL-03	La CURP ya cuenta con un proceso vigente.	ACTIVE_PROCESS_EXISTS - Ya existe un trámite activo para esta CURP.	409
VAL-04	Recurso solicitado inexistente.	RESOURCE_NOT_FOUND - No se encontró el registro solicitado.	404
VAL-05	Archivo vacío, corrupto o con tipo no permitido.	INVALID_DOCUMENT - Use PDF, JPG o PNG.	400/415
VAL-06	Archivo mayor al límite configurable de 10 MB.	DOCUMENT_TOO_LARGE - El archivo supera el tamaño permitido.	413
VAL-07	Documento no observado durante la subsanación.	DOCUMENT_NOT_OBSERVED - Solo puede reemplazar documentos señalados.	409
VAL-08	Dictamen incompatible con el estado actual.	INVALID_STATUS_TRANSITION - No es posible aplicar el dictamen solicitado.	409
VAL-09	Enrolamiento desde expediente no aceptado.	EXPEDIENT_NOT_ACCEPTED - El expediente no está aprobado.	409
VAL-10	El expediente ya originó un alumno.	STUDENT_ALREADY_EXISTS - El expediente ya tiene matrícula.	409
VAL-11	Colisión de matrícula después de los reintentos internos.	ENROLLMENT_GENERATION_FAILED - No fue posible generar la matrícula.	500
VAL-12	Movimiento académico redundante o no permitido.	INVALID_STUDENT_STATUS - El cambio de estado no procede.	409
VAL-13	Rol Coordinación sin carrera asignada.	CAREER_REQUIRED - Debe asignarse una carrera al coordinador.	400
VAL-14	Matrícula de empleado duplicada.	EMPLOYEE_ID_EXISTS - La matrícula ya está registrada.	409
VAL-15	Periodo inexistente o inactivo.	PERIOD_UNAVAILABLE - El periodo no admite operaciones.	404/409
VAL-16	Alumno inactivo o no apto para inscripción.	STUDENT_NOT_ELIGIBLE - El estado actual impide la inscripción.	409
VAL-17	Grupo correspondiente a otra carrera.	CAREER_MISMATCH - El grupo no pertenece a la carrera del alumno.	409
VAL-18	Grupo sin disponibilidad.	GROUP_FULL - El grupo alcanzó su capacidad máxima.	409
VAL-19	Inscripción duplicada para el mismo grupo y periodo.	DUPLICATE_ENROLLMENT - La inscripción ya existe.	409
VAL-20	Conversación ajena o fuera del ámbito autorizado.	CONVERSATION_FORBIDDEN - No tiene acceso al hilo.	403
VAL-21	Mensaje vacío o conversación cerrada.	MESSAGE_INVALID / CONVERSATION_CLOSED.	400/409
VAL-22	Token ausente, inválido o expirado.	UNAUTHORIZED - La sesión no es válida o expiró.	401
VAL-23	Rol válido sin permiso o fuera de su carrera.	FORBIDDEN - No cuenta con autorización para esta operación.	403
VAL-24	Error interno no controlado.	INTERNAL_ERROR - No fue posible completar la operación.	500

Para los códigos combinados, la API distinguirá la causa: 404 cuando el recurso no exista; 409 cuando exista pero su estado impida continuar; 400 cuando la entrada esté mal formada; y 415 cuando el tipo de archivo no sea compatible.

4.5 Diseño de autenticación y permisos (SCRUM-23)

4.5.1 Inicio de sesión y emisión de JWT

Tabla 4.6. Flujo de autenticación.

Paso	Comportamiento
1	El usuario envía matrícula y contraseña mediante POST /api/auth/login.
2	La API identifica si la matrícula pertenece a alumno o empleado.
3	La contraseña se compara con el hash almacenado; nunca se recupera ni se devuelve.
4	Para alumnos se comprueba que el expediente no esté cerrado de forma definitiva.
5	La API firma un JWT con vigencia configurable y devuelve datos mínimos de sesión.

La duración del token se controlará mediante JWT_EXPIRES_IN; no se codificará una vigencia fija dentro de las rutas. La matrícula sirve como credencial de acceso, mientras que la identidad de sesión se obtiene del claim sub del JWT.

4.5.2 Contenido mínimo del token

Tabla 4.7. Claims autorizados en el JWT.

Claim	Tipo	Uso
sub	Entero	Identificador del alumno o empleado autenticado.
tipoUsuario	Cadena	ALUMNO o EMPLEADO.
rol	Cadena	ALUMNO, ADMISIONES, CONTROL_ESCOLAR o COORDINACION.
carreraId	Entero opcional	Solo cuando el ámbito del usuario depende de una carrera.
iat / exp	Númérico	Fechas de emisión y expiración administradas por JWT.

No se incluirán contraseñas, hashes, CURP, documentos ni información personal innecesaria dentro del token.

4.5.3 Matriz de control de acceso

Tabla 4.8. Permisos por actor y ámbito.

Actor / rol	Operaciones	Restricción
Público	Login, carreras activas y pre-registro.	No permite consultar datos institucionales.
Aspirante	Estado propio, documentos y subsanación.	Acceso mediante token temporal asociado con un solo expediente.
Admisiones	Aspirantes, expedientes, documentos y dictamen.	No genera matrícula ni administra grupos.
Control Escolar	Alumnos, empleados, periodos, grupos, inscripciones, historial y reportes.	Ámbito administrativo general aprobado.
Coordinación	Alumnos, grupos, inscripciones y conversaciones.	Limitado a la carrera obtenida del JWT.
Alumno	Perfil e inscripción propios; conversaciones y mensajes propios.	No puede modificar expediente, estado, grupo o datos de terceros.

4.5.4 Protección de rutas

- authenticateJWT validará el encabezado Authorization: Bearer <token> y adjuntará la identidad autenticada a la petición.
- checkRole verificará los roles autorizados antes de ejecutar el controlador.
- La carrera de Coordinación se obtendrá del JWT; el cliente no podrá ampliar su ámbito mediante parámetros de consulta.
- El remitente de un mensaje y el alumno consultado en rutas /me se derivarán de la sesión, no del cuerpo enviado.
- 401 indicará ausencia o invalidez de autenticación; 403 indicará identidad válida sin permiso suficiente.

4.5.5 Tratamiento de credenciales

Las contraseñas se almacenarán únicamente como hashes seguros. Ninguna respuesta, reporte o correo incluirá la contraseña almacenada. Al finalizar el enrolamiento, el aspirante podrá recibir su matrícula e instrucciones para establecer o cambiar su contraseña mediante un mecanismo temporal.

4.6 Documentar y revisar la API (SCRUM-24)

Estado: concluido. Las rutas aprobadas se reunieron en una colección inicial de Postman y se revisaron contra las UFP, los contratos, las validaciones y los permisos definidos en los apartados 4.1 a 4.5.

4.6.1 Colección inicial de Postman

La colección se conserva en docs/postman/AUT-INS_API.postman_collection.json y utiliza el formato Postman Collection v2.1. Su propósito es reunir los contratos de diseño en un recurso importable y preparar la ejecución de pruebas cuando la API esté implementada.

Tabla 4.9. Evidencias de cierre de SCRUM-24.

Elemento	Evidencia integrada	Resultado
Formato	Colección JSON compatible con Postman v2.1.	Conforme
Contratos	35 combinaciones únicas de método y ruta.	Conforme
Organización	13 carpetas agrupadas por capacidad funcional.	Conforme
Variables	13 variables para URL, tokens e identificadores reutilizables.	Conforme
Seguridad	Acceso público, token temporal y JWT según la operación.	Conforme
Solicitudes	Cuerpos JSON, formularios, filtros y parámetros de ejemplo.	Conforme
Pruebas iniciales	Códigos HTTP, JSON, tiempo de respuesta y captura de datos.	Conforme
Reproducibilidad	Comandos postman:build y check:postman en el repositorio.	Conforme

4.6.2 Cobertura de las Unidades Funcionales de Proceso

La cobertura se comprobó relacionando cada UFP con al menos una solicitud de la colección. Los módulos académicos complementarios se conservaron sin condicionar el flujo principal de admisión e inscripción.

Tabla 4.10. Matriz de cobertura UFP-Postman.

UFP	Rutas o carpetas que proporcionan cobertura	Resultado
UFP-01	Carreras, Aspirantes y carga de Documentos.	Cubierta
UFP-02	Expedientes, Dictamen y revisión de Documentos.	Cubierta
UFP-03	Enrolamiento de Alumnos e Inscripciones.	Cubierta
UFP-04	Alumnos, Personal, Expedientes e Historial.	Cubierta
UFP-05	Periodos, Grupos, Inscripciones y Materias.	Cubierta
UFP-06	Perfil e inscripción propios del Alumno.	Cubierta
UFP-07	Conversaciones, Mensajes, Lectura y Estado.	Cubierta
UFP-08	Historial por CURP y Reportes.	Cubierta
UFP-SUG-01	Grupos e Inscripciones con validación de cupo.	Cubierta
UFP-SUG-02	Login, JWT y permisos de las rutas.	Cubierta
UFP-SUG-03	Estado del trámite y sustitución documental.	Cubierta

4.6.3 Revisión conjunta del diseño

La revisión conjunta se consolidó mediante una comprobación cruzada de los entregables de la etapa. Esta revisión valida el diseño documental; la ejecución completa y sus evidencias pertenecen a la sexta etapa.

Tabla 4.11. Lista de comprobación de la revisión.

Criterio	Comprobación realizada	Resultado
Consistencia	La colección coincide con las rutas del apartado 4.2.	Conforme
Contratos	Los cuerpos y salidas respetan el apartado 4.3.	Conforme
Errores	Los códigos esperados consideran las reglas del apartado 4.4.	Conforme
Permisos	Cada solicitud declara acceso público, temporal o JWT.	Conforme
Identidad	Los datos de sesión no se aceptan del cliente cuando pueden derivarse del JWT.	Conforme
Cobertura	Las once UFP cuentan con rutas asociadas.	Conforme
Integridad	No existen combinaciones duplicadas de método y ruta.	Conforme

4.6.4 Conclusión de la etapa

Resultado: aprobado. El diseño de la API es consistente con el alcance acordado, cubre todas las UFP y cuenta con una colección inicial importable. Con estas evidencias se cierra el sprint de diseño y se autoriza el inicio de la quinta etapa.

4.7 Correspondencia entre el diseño y la implementación actual

Las 35 rutas de los apartados anteriores representan el contrato aprobado y la colección inicial de Postman. Al cierre de esta revisión, el servidor Express publica la ruta de diagnóstico y monta los prefijos modulares, pero los routers de negocio todavía no contienen controladores operativos. Esta distinción evita presentar como ejecutado lo que aún es diseño.

Tabla 4.12. Estado real de las rutas en el código.

Superficie	Estado	Comportamiento comprobado
GET /	Implementado	Entrega public/index.html y los recursos del front demostrativo.
GET /api/health	Implementado	Devuelve estado, versión, fecha y ocho módulos registrados.
/api/auth, /aspirantes, /admisiones, /control-escolar	Montados	Existen routers modulares; sus endpoints de negocio siguen pendientes.
/api/coordinacion, /alumnos, /mensajes, /historial	Montados	Existen routers modulares; sus endpoints de negocio siguen pendientes.
Rutas no definidas	Implementado	El middleware notFound devuelve una respuesta JSON 404 uniforme.

5. Quinta etapa. Desarrollo

Estado de la etapa. *Desarrollo parcial controlado. La base ejecutable, la persistencia y la demostración web están disponibles; la lógica de los endpoints de negocio, JWT y EmailJS aún debe implementarse para considerar terminado el backend funcional.*

5.1 Objetivo y criterio de implementación

El desarrollo se organizó por módulos para que cada área avance sin redefinir el modelo de datos ni invadir responsabilidades. El servidor, la base y los contratos constituyen una línea base estable; cualquier cambio de requisito debe registrarse antes de modificar el esquema o la colección.

5.2 Arquitectura del repositorio

Tabla 5.1. Estructura técnica implementada.

Ubicación	Responsabilidad
src/app.ts	Configura Express, CORS, límites JSON, archivos estáticos, /api y manejo de errores.
src/server.ts	Inicia el servicio y realiza un cierre controlado ante SIGINT o SIGTERM.
src/config	Centraliza variables de entorno y el cliente Prisma.
src/modules	Separa auth, aspirantes, admisiones, Control Escolar, Coordinación, alumnos, mensajes, historial y common.
prisma	Contiene esquema, migración, semilla y documentación de persistencia.
public	Contiene la interfaz web demostrativa y sus recursos.
docs/postman	Contiene la colección inicial con los contratos de la API.
scripts	Incluye verificadores, generación de Postman y preparación local de MySQL.

5.3 Módulos y límites funcionales

Tabla 5.2. Estado de los módulos del backend.

Módulo	Alcance	Estado actual
auth	Login, renovación, cierre y permisos JWT.	Router creado; lógica pendiente.
aspirantes	Pre-registro, consulta y subsanación.	Router creado; lógica pendiente.
admisiones	Bandeja, documentos y dictamen.	Router creado; lógica pendiente.
control-escolar	Enrolamiento, matrícula, altas, bajas y correcciones.	Router creado; lógica pendiente.
coordinacion	Grupos, capacidad y asignaciones por carrera.	Router creado; lógica pendiente.
alumnos	Perfil, estado e inscripción propios.	Router creado; lógica pendiente.
mensajes	Conversaciones, envío y lecturas.	Router creado; lógica pendiente.
historial	Procesos históricos por CURP.	Router creado; lógica pendiente.
common	Salud, errores y rutas inexistentes.	Implementado y operativo.

5.4 Componentes desarrollados

Tabla 5.3. Resultado del desarrollo actual.

Componente	Resultado	Estado
Servidor	Express 5, CORS configurable, límites de entrada, publicación estática y cierre controlado.	Operativo
Diagnóstico	GET /api/health con versión y listado de módulos.	Operativo
Errores	Respuesta uniforme para AppError, errores internos y rutas inexistentes.	Operativo
Persistencia	Cliente Prisma, 25 tablas, migración inicial y datos semilla.	Implementado
Contratos	Colección Postman v2.1 con 35 solicitudes y once UFP.	Implementado
Front	Landing, solicitud, login y paneles de cuatro roles con datos de demostración.	Demostración funcional
JWT	Diseño de claims, vigencia y permisos.	Pendiente en backend
EmailJS	Alcance y tipos de notificación definidos.	Pendiente de integración
Almacenamiento documental	Modelo de metadatos y versionado definido.	Proveedor de archivos pendiente

5.5 Configuración y comandos

Tabla 5.4. Comandos de desarrollo y ejecución.

Comando	Propósito
npm install	Instala dependencias y genera el cliente Prisma.

Comando	Propósito
npm run mysql:setup	Prepara MySQL local aislado en el puerto 3307 y genera .env.
npm run db:deploy	Aplica las migraciones existentes.
npm run db:seed	Carga o actualiza los catálogos iniciales.
npm run dev	Ejecuta el servidor con recarga durante el desarrollo.
npm run check	Valida esquema, tipos, estructura, Postman y compilación estática.
npm run build / npm start	Compila y ejecuta dist/server.js.

5.6 Consideraciones de despliegue

- Render puede ejecutar el servicio Node.js mediante npm run build y npm start.
- Azure alojará únicamente MySQL y deberá permitir conexiones desde el servicio de la API.
- DATABASE_URL, CORS_ORIGIN y cualquier secreto deben definirse como variables privadas del entorno.
- En producción se ejecuta prisma migrate deploy; migrate dev queda reservado al desarrollo.
- La salud del servicio se comprueba mediante GET /api/health.

Límite actual. La compatibilidad de despliegue está preparada a nivel de scripts y servidor, pero no existe evidencia de una instancia productiva publicada ni de una conexión Azure configurada. El documento no la presenta como completada.

6. Sexta etapa. Pruebas

Responsable registrado: Pedro Jair Suárez Flores. Esta revisión documenta las comprobaciones disponibles al 12 de agosto de 2026 y separa las validaciones ejecutadas de los casos que requieren la implementación futura de los endpoints de negocio.

6.1 Resumen de validaciones ejecutadas

Tabla 6.1. Validaciones automatizadas ejecutadas.

Área	Ejecución	Resultado observado	Estado
Prisma	npm run db:validate	Esquema válido.	Conforme
Tipos de base	npm run db:typecheck	Configuración y semilla sin errores de TypeScript.	Conforme
Estructura	npm run check:structure	9 módulos y 17 archivos TypeScript reconocidos.	Conforme
Postman	npm run check:postman	35 rutas únicas y 11 UFP cubiertas.	Conforme
Tipos del backend	npm run typecheck	Sin errores de tipado.	Conforme
Compilación	npm run build	dist generado correctamente.	Conforme

6.2 Pruebas de humo HTTP

Tabla 6.2. Resultados de las pruebas de humo locales.

Solicitud	HTTP	Contenido	Comprobación
GET /api/health	200	application/json	Servicio y módulos disponibles.
GET /	200	text/html	Interfaz principal entregada por Express.
GET /api/aspirantes	404	application/json	Resultado esperado: router sin endpoint implementado.
GET /api/ruta-inexistente	404	application/json	Middleware de ruta inexistente operativo.

6.3 Revisión manual del front

Tabla 6.3. Resumen de prueba manual del front.

Escenario	Evidencia observada	Resultado
Navegación pública	Landing, solicitud e inicio de sesión visibles y navegables.	Conforme
Admisiones	Bandeja de solicitudes, filtros y revisión de expediente disponibles.	Conforme en demostración
Control Escolar	Enrolamiento, alumnos, cambio de estado, acceso y PDF disponibles.	Conforme en demostración
Coordinación	Grupos, capacidad, asignación y bandeja personal disponibles.	Conforme en demostración
Alumno	Perfil, inscripción y caja de mensajes con selección de destinatario disponibles.	Conforme en demostración
Sesión	Configuración de 10 segundos cerró la sesión por inactividad y regresó al login.	Conforme
Consola	No se detectaron errores durante el recorrido realizado.	Conforme

6.4 Alcance de la colección Postman

La colección incluye aserciones iniciales de código HTTP, formato JSON, tiempo de respuesta y captura de identificadores. Su estructura fue validada automáticamente; sin embargo, la ejecución de los 35 flujos contra la API real permanece pendiente porque los routers de negocio aún no implementan esos contratos.

6.5 Casos funcionales pendientes

- Login válido, inválido, expirado y acceso con rol incorrecto mediante JWT real.
- Pre-registro correcto, formatos inválidos y bloqueo por proceso activo de la CURP.
- Carga, observación y sustitución documental con almacenamiento real.
- Dictamen aceptado, rechazado y observado con historial y EmailJS.
- Enrolamiento transaccional, matrícula única y cambio obligatorio de contraseña.
- Inscripción con cupo, sobrecupo, grupo de otra carrera y periodo cerrado.
- Mensajería autorizada, conversación ajena, lectura y cierre.

- Consultas históricas por CURP y aislamiento de datos por rol y carrera.

Conclusión de pruebas. *La línea base compila, el esquema y los artefactos documentales son consistentes, y la demostración web completa el recorrido visual. No puede declararse aprobada la API de negocio hasta ejecutar los casos funcionales anteriores contra controladores y una base de pruebas.*

7. Séptima etapa. Front end

Estado de la etapa. *Demostración funcional concluida; integración con la API real pendiente. Responsable previsto: Carlos Eduardo Martínez Morales.*

7.1 Propósito y enfoque visual

La interfaz presenta un tema claro con fondo gris azulado, acentos institucionales, tarjetas y paneles responsivos. Bootstrap se utiliza para la estructura y los componentes, mientras que CSS propio unifica la identidad AUT-INS y adapta las vistas a escritorio y móvil.

7.2 Vistas y funciones disponibles

Tabla 7.1. Componentes del front implementado.

Vista	Alcance demostrativo
Inicio	Explica el flujo, ofrece acceso a solicitud y login, y presenta las cuatro experiencias principales.
Solicitud	Captura datos, CURP, carrera, contacto, consentimiento y documentación de demostración.
Login	Ofrece accesos discretos de prueba y redirige según el rol identificado.
Admisiones	Métricas, búsqueda, filtro, expediente, documentos, observaciones y dictamen demostrativo.
Control Escolar	Enrolamiento, matrícula, contraseña temporal, PDF, alumnos, estados y mensajes.
Coordinación	Catálogo de grupos, capacidad, activación, asignación compatible y mensajes directos.
Alumno	Matrícula, estado, carrera, grupo, periodo, turno y caja de mensajes.

7.3 Sesión y separación por roles

La sesión demostrativa se guarda en sessionStorage. Si el usuario intenta abrir un panel sin sesión, regresa al login; si intenta entrar al panel de otro rol, se redirige al panel autorizado. El tiempo de inactividad es configurable entre 10 segundos y 30 minutos para facilitar la demostración, con 15 minutos como valor recomendado.

Importante. *Este comportamiento simula la sesión en el navegador. La seguridad definitiva depende de JWT, validación de expiración en el servidor, hashes de contraseña y autorización en cada endpoint.*

7.4 Expediente y enrolamiento demostrativo

Admisiones puede abrir los datos y la relación de documentos enviados por cada aspirante. Control Escolar recibe los expedientes aceptados y puede generar una matrícula y una contraseña temporal. El front permite descargar un PDF confidencial de acceso mediante jsPDF y solicita que la credencial se cambie en el primer ingreso.

7.5 Mensajería interna

El alumno elige entre el departamento de Control Escolar y un coordinador específico. Los mensajes dirigidos al departamento aparecen para los usuarios que comparten ese rol; los mensajes a Coordinación se muestran únicamente en la bandeja personal del coordinador seleccionado.

7.6 Persistencia demostrativa y límites

- Los datos de prueba se guardan localmente en el navegador y pueden restablecerse.
- Los documentos cargados se representan mediante metadatos; el almacenamiento real aún no está conectado.
- Las credenciales visibles son exclusivas del entorno de demostración y no deben reutilizarse.
- Las acciones del front no modifican todavía MySQL porque los endpoints de negocio siguen pendientes.
- EmailJS no se ejecuta desde esta demostración; solamente está definido el flujo esperado.

7.7 Criterios para la integración definitiva

Tabla 7.2. Pendientes para conectar el front con la API.

Área	Trabajo de integración
Autenticación	Reemplazar demo-store por POST /api/auth/login y almacenar el JWT conforme a la estrategia de seguridad.
Datos	Sustituir arreglos de demostración por llamadas a los endpoints autorizados.
Archivos	Integrar carga multipart y previsualización desde un proveedor de almacenamiento seguro.
Errores	Mostrar códigos y mensajes uniformes de la API sin exponer detalles internos.
Sesión	Coordinar la inactividad del cliente con exp e invalidación del JWT.
Pruebas	Ejecutar los mismos recorridos con datos aislados en un entorno de pruebas.

8. Octava etapa. Evidencias y entrega

La entrega actual reúne el código fuente, el modelo físico, la migración, los datos semilla, la colección de Postman, la documentación Markdown, la interfaz demostrativa y este documento maestro.

8.1 Inventario de evidencias

Tabla 8.1. Inventario de la entrega.

Evidencia	Ubicación	Contenido
Código fuente	src/, public/, scripts/	Repositorio privado de GitHub
Base de datos	prisma/schema.prisma y migrations/	Modelo y migración reproducibles
Catálogos	prisma/seed.ts	Carga idempotente
Diseño de API	docs/postman/AUT-INS_API.postman_collection.json	35 solicitudes y 11 UFP

Evidencia	Ubicación	Contenido
Documentación técnica	docs/*.md y README.md	Alcance, tecnología, datos, API y Scrum
Front	public/index.html y public/assets/	Demostración por roles
Documento académico	Proyecto_Aplicacion_AUT-INS_DOCUMENTACION_FINAL.docx	Memoria integral del proyecto

8.2 Instalación resumida

```
git clone https://github.com/jr-0205/aut-ins-api.git
cd aut-ins-api
npm install
npm run mysql:setup
npm run db:deploy
npm run db:seed
npm run dev
```

Después de iniciar el servidor, la interfaz está disponible en <http://localhost:3000> y el diagnóstico en <http://localhost:3000/api/health>.

8.3 Repositorio y control de acceso

Repositorio del proyecto: [Repositorio privado AUT-INS en GitHub](#)

El repositorio es privado. Solamente los colaboradores invitados pueden consultar el avance. No deben publicarse archivos .env, tokens, claves de Azure, documentos personales ni contraseñas de alumnos.

8.4 Estado general de cierre documental

Tabla 8.2. Estado verificable del proyecto.

Etapas	Estado	Justificación
Etapas 1 y 2	Concluidas	Actores, UFP, cadena de valor y decisiones tecnológicas consolidadas.
Etapas 3	Concluida	Modelo normalizado, esquema, migración y catálogos implementados.
Etapas 4	Concluida	Contrato de API y colección inicial revisados.
Etapas 5	Parcial	Base técnica operativa; endpoints de negocio pendientes.
Etapas 6	Parcial	Validaciones técnicas y front ejecutados; pruebas funcionales de API pendientes.
Etapas 7	Demostración concluida	Interfaz por roles disponible; integración real pendiente.
Etapas 8	Concluida para esta entrega	Evidencias e instalación reunidas.

Conclusiones

AUT-INS cuenta con un análisis funcional consistente, un modelo relacional normalizado y reproducible, un contrato de API amplio, una base modular compilable y una interfaz demostrativa que permite recorrer el proceso por roles. La estructura resuelve la principal preocupación del proyecto: conservar la identidad e historial de una CURP sin permitir procesos vigentes duplicados.

La revisión también establece un límite importante: la existencia de 35 rutas en Postman no significa que los controladores estén terminados. El estado real del backend se documenta como base ejecutable con diagnóstico, persistencia y routers modulares, mientras que el front opera con datos locales de demostración. Esta separación permite continuar el desarrollo sin generar expectativas incorrectas.

El siguiente incremento debe concentrarse en implementar el flujo completo de admisión e inscripción de extremo a extremo: autenticación, pre-registro, expediente, dictamen, enrolamiento, inscripción, mensajes e historial. Después deberán ejecutarse las pruebas funcionales de Postman y conectar el front a la API.

Anexo A. Glosario

Tabla A.1. Glosario del proyecto.

Término	Definición
Aspirante	Persona que inicia un intento de admisión y todavía no tiene una cuenta de alumno activa.
Expediente	Conjunto lógico de folio, estado, dictamen y documentos de un intento de admisión.
Enrolamiento	Conversión controlada de un expediente aceptado en alumno y cuenta institucional.
Proceso activo	Único intento vigente asociado con una persona; bloquea otro registro con la misma CURP.
UFP	Unidad Funcional de Proceso que agrupa una capacidad del sistema.
JWT	Token firmado que identifica al usuario y limita la vigencia de la sesión.
ORM	Capa que representa las tablas y relaciones mediante modelos de código.
Migración	Script versionado que reproduce una modificación del esquema físico.
Dato semilla	Registro inicial controlado, como un rol o estado de catálogo.

Anexo B. Comandos de verificación

```
npm run db:validate
npm run db:typecheck
npm run check:structure
npm run check:postman
npm run typecheck
npm run build
npm start
```

La prueba mínima de disponibilidad se realiza con GET /api/health. Para pruebas completas se deberá importar docs/postman/AUT-INS_API.postman_collection.json y configurar baseUrl, tokens e identificadores del entorno de pruebas.

Anexo C. Trazabilidad Scrum

Tabla C.1. Relación entre el tablero y el documento.

Tarea	Entregable	Ubicación
SCRUM-5	Identificación de actores	Integrado en 1.1
SCRUM-6	Matriz de actores y responsabilidades	Integrado en 1.2
SCRUM-7	Unidades Funcionales de Proceso	Integrado en 1.3 y 1.4
SCRUM-8	Cadena de valor	Integrado en 1.5
SCRUM-9	Diseño de solución y tecnologías	Integrado en etapa 2
SCRUM-13	Análisis de requisitos y entidades	Integrado en 3.3 a 3.9
SCRUM-14	Primera Forma Normal	Integrado en 3.11
SCRUM-15	Segunda Forma Normal	Integrado en 3.12
SCRUM-16	Tercera Forma Normal	Integrado en 3.13
SCRUM-17	Diseño e implementación de base	Integrado en 3.14
SCRUM-19 a 24	Diseño y revisión de API	Integrado en etapa 4
Desarrollo	Base técnica y front demostrativo	Integrado en etapas 5 y 7
Pruebas	Validaciones ejecutadas y plan pendiente	Integrado en etapa 6